UFM · WORKSHOP DE PRODUCTO Y CONSTRUCCIÓN CON IA

MANUAL DEL ESTUDIANTE + CONTEXTO PARA AGENTES DE CÓDIGO

BEYOND VIBE CODING.

De entrevistas y un diseño interactivo a un MIU funcional en internet.

Cuatro talleres para diseñar con criterio, construir con IA, publicar con Next.js + Supabase + Vercel y mejorar a partir de usuarios reales.

INICIO **Lunes 26 oct.**	CONSTRUCCIÓN **Miércoles 28 oct.**
VALIDACIÓN **Lunes 2 nov.**	DEMO FINAL **Miércoles 4 nov.**

Universidad Francisco Marroquín
Estudiantes de primer año · Guatemala

Versión 1.0
25 de octubre de 2026

01 · Cómo usar este documento

Este material funciona de **dos maneras**.

1. **Manual del estudiante.** Sigue las instrucciones, marca los hitos y utiliza los videos y documentación cuando una pantalla cambie o algo falle.
2. **Contexto para el agente de código.** Sube el PDF a Claude Code, Codex, Cursor o Antigravity, o pega su archivo Markdown complementario. Añade después el documento con el research de aproximadamente **30 entrevistas** y el HTML/ZIP del diseño. Pide primero un plan y **aprueba ese plan antes de autorizar cambios**.

Orden de prioridad de la información: investigación real → decisiones del equipo → diseño/prototipo exportado → esta guía → sugerencias de la IA. La IA puede proponer mejoras, pero no debe inventar hallazgos ni simular integraciones universitarias reales.

Glosario esencial

Término	Qué significa aquí
Insight	Comprensión concreta del problema detrás de lo observado en una entrevista.
MVP	Primera versión pequeña pero útil que permite terminar tareas reales dentro de la demostración.
Design system	Reglas reutilizables de color, tipografía, componentes, estados y espaciado.
Happy path	Camino ideal, cuando todo funciona según lo esperado.
Edge case	Situación excepcional: tarea vacía, usuario sin acceso, dato duplicado, error de red, etc.
Frontend / backend	Pantallas e interacciones / lógica, autenticación y persistencia de datos.
Deployment	Publicación de la aplicación en una URL accesible desde internet.
OAuth	Protocolo por el cual Google verifica quién eres; no equivale a acceder al MIU oficial.
RLS	Reglas de Supabase que restringen qué registros puede leer o modificar cada usuario.

02 · El mapa de los cuatro workshops

Ritmo general: aproximadamente 30 minutos para conceptos/demostración y 30 para ejecución guiada. La primera sesión reserva 15 minutos para la presentación del estudiante sobre su Grokbot, y la tercera prioriza investigación de campo.

Fecha / workshop	Tiempo en clase	Objetivo de salida	Trabajo autónomo obligatorio
Lun. 26 oct. · Diseña antes de programar	00–15: Grokbot; 15–30: insight, buen gusto, sistema de diseño y happy paths; 30–60: construir diseño y primeros flujos.	Design system definido, mínimo 3 flujos dibujados/ interactivos; **exportar HTML independiente o ZIP**; solicitar **solo el plan de implementación**.	Terminar Claude Design y exportación antes del miércoles; leer las entrevistas.
Mié. 28 oct. · Del prototipo al código	00–30: arquitectura, cuentas, handoff, GitHub, Supabase, Vercel, OAuth; 30–60: laboratorio en parejas/ equipos.	Repositorio y proyecto local funcionando; Supabase creado; construcción iniciada; primer despliegue si es posible.	Finalizar flujos, conectar base de datos, publicar versión usable con Google Login antes del lunes.
Lun. 2 nov. · El usuario decide	00–10: briefing y tareas; 10–45: 3 pruebas reales por equipo; 45–60: comparar feedback y elegir mejoras.	URL accesible, tres observaciones documentadas, patrones de fricción y 2–3 mejoras priorizadas.	Implementar los cambios de mayor valor, verificar y volver a publicar.
Mié. 4 nov. · Demo day	00–10: preparación; 10–50: presentaciones cronometradas; 50–60: síntesis y aprendizajes.	Producto publicado, evidencia de mejora, presentación, README y URL final.	Entrega completa ese día.

Ajuste de tiempo: si hay muchos equipos, el demo debe ser de 2–3 minutos por equipo o trasladar parte de las demostraciones a revisión de enlaces; reservar tiempo para observar el producto, no solo escuchar una presentación.

Cinco puertas de calidad

- **G1 — Problema:** puedo decir quién sufre qué fricción y qué evidencia lo respalda.
- **G2 — Diseño:** el sistema visual es coherente, responsive y los tres happy paths son verificables.
- **G3 — Construcción:** los componentes importantes no son botones decorativos; hacen algo y guardan/leen datos cuando corresponde.
- **G4 — Validación:** tres estudiantes completaron tareas en un enlace público y recibimos feedback observado, no solo opiniones.
- **G5 — Entrega:** el equipo corrigió al menos dos hallazgos y explica sus trade-offs.

03 · Enfocar el problema antes de abrir la IA

La clase anterior terminó con una investigación de discovery: aproximadamente **30 entrevistas a estudiantes** sobre tareas, clases, horarios, profesores y comunicación interna. **El documento de entrevistas será la fuente primaria**, y aún debe ser leído por cada equipo. No se atribuye aquí ningún hallazgo específico a esas entrevistas.

Cómo transformar las entrevistas en decisiones

1. **Leer sin diseñar:** subrayar frases que describen acciones, frustraciones y consecuencias.
2. **Agrupar:** usar categorías como claridad de fechas, seguimiento de tareas, carga académica, horarios, comunicación, recordatorios o descubrimiento de información. Son categorías orientativas, no hallazgos confirmados.
3. **Contar evidencia:** ¿cuántos entrevistados mencionaron el problema?, ¿con qué intensidad?, ¿qué solución improvisada usan hoy?
4. **Redactar el insight:** «Cuando [contexto], [usuario] necesita [resultado], porque [obstáculo/causa]».
5. **Priorizar:** impacto para el alumno × recurrencia × capacidad de prototipar en una semana; descartar lo accesorio.
6. **Convertir en historia de usuario y criterio de aceptación:** «Como estudiante quiero ver mis entregas ordenadas por fecha para anticipar mi semana». Aceptación: desde el dashboard encuentro la próxima entrega, su curso y su fecha sin preguntar dónde está.

La regla: nunca empezar con «hagamos una plataforma con 25 funciones». Empezar con «qué problema podemos solucionar tan bien que tres compañeros lo noten».

Mini-scorecard de priorización (0–3)

Criterio	Pregunta
Evidencia	¿Lo dijeron varios alumnos o fue un único caso?
Intensidad	¿La fricción provoca pérdida de tiempo, errores o estrés?
Frecuencia	¿Sucede todas las semanas o una vez al año?
Viabilidad	¿Podemos resolverlo con datos ficticios y una app real en cuatro sesiones?

Elegir 1 problema principal y hasta 2 secundarios. **La experiencia MIU no debe convertirse en una colección de funcionalidades aisladas.**

04 · ¿Qué es un buen producto cuando todos pueden generar código?

El diferencial ya no es producir pantallas rápidamente. Es tener criterio. La calidad se demuestra en decisiones observables: un problema priorizado, jerarquía visual, navegación simple, textos claros, accesibilidad, estados bien pensados y flujos completos.

Cómo desarrollar buen gusto

- Explorar **3–5 referencias** de dashboards de educación, calendarios, tareas y productos con excelente UX: Dribbble, Behance, Pinterest, Figma Community.
- Capturar **2–4 screenshots** que expliquen decisiones visuales: densidad, tarjetas, tablas, filtros, formularios, sidebar, versión móvil.
- Anotar junto a cada captura **qué problema visual resuelve**. Evitar copiar la marca, el producto completo o assets ajenos; inspirarse en patrones, no clonar.
- Dar instrucciones específicas a Claude Design: «prioriza lectura rápida del calendario», «muestra vencimientos con contraste suficiente», «evita gradientes y tarjetas idénticas en cada sección».
- Validar tamaños de texto, contraste, estados hover/focus, mensajes de éxito/error y adaptación a teléfono.

Un sistema de diseño mínimo debe incluir: colores semánticos; familia y escala tipográfica; escala de espaciado (p. ej. 4/8 px); radio de bordes; botones, inputs, selects, tablas, tarjetas, badges y navegación; iconografía coherente; estados vacío/cargando/error/éxito; criterios para móvil.

Pantallas recomendadas para el MVP

1. **Dashboard:** próximas clases, tareas prioritarias y anuncios relevantes.
2. **Mis tareas:** listado, filtros, detalle y acción significativa (marcar avance o enviar una entrega ficticia).
3. **Horario:** ver clases por día/semana y abrir información de curso/profesor.
4. **Comunicaciones:** anuncios del curso, ordenados por fecha, como funcionalidad opcional si está en los insights.

La lista anterior es una **propuesta inicial**, que debe cambiar según el documento real de research.

05 · Diseñar los happy paths con intención

En esta entrega modelaremos **al menos tres happy paths completos**. No se requiere replicar reglas administrativas de una universidad real.

Flujo	Inicio → final	Señal de que funciona
A. Próxima entrega	Inicio → Mis tareas → Detalle → reconocer fecha/estado → registrar avance.	El estudiante encuentra y actualiza su información; el cambio persiste tras refrescar.
B. Próxima clase	Inicio → Horario → fecha → clase → información de aula/profesor.	Puede decir dónde y cuándo debe presentarse.
C. Comunicación académica	Inicio → Anuncios → anuncio de un curso → contenido.	Encuentra un aviso importante sin explorar pantallas innecesarias.

Happy path no significa ignorar todo lo demás. Añadir al menos los estados técnicos mínimos: sin tareas, cargando, solicitud fallida, sesión caducada, guardado exitoso y acceso no autorizado. Otros edge cases (bloqueo por pago, reglas de reinscripción, integraciones de parqueo, identidad institucional) se documentan como **fuera de alcance**.

Diseño interactivo ≠ aplicación terminada

En Claude Design, un botón puede demostrar un recorrido aunque tenga datos simulados. En el producto final, cada interacción evaluada debe ser real dentro del prototipo: navegación válida, sesión de usuario, lectura/escritura de datos de demostración y estado persistente. Un HTML independiente **no es por sí mismo** una app conectada a Supabase.

Prompt 1 · Del research a decisiones de producto

```
Actúa como Product Manager y UX Researcher senior.
Te adjunto entrevistas reales a estudiantes que usan MIU.
1) No inventes citas, cifras ni hallazgos.
2) Extrae pain points verificables con evidencia y frecuencia
   SOLO cuando el documento permita contarlas.
3) Agrupa por Job To Be Done y distingue problema de solución.
4) Prioriza los 3 problemas con mayor impacto y viabilidad.
5) Propón 3 historias de usuario con criterios de aceptación.
6) Enumera dudas o hipótesis todavía no validadas.
Salida: tabla evidencia → insight → oportunidad → flujo.
Contexto: tenemos 4 workshops y haremos un prototipo
académico independiente, NO una integración oficial de UFM.
```

Prompt 2 · Sistema de diseño en Claude Design

```
Crea el sistema de diseño para «MIU Reimagined», una propuesta
NO OFICIAL de experiencia universitaria para estudiantes UFM.
Adjunto 2–4 referencias visuales y una lista de insights reales.
Usa las referencias como inspiración, nunca como copia exacta.
Define tokens de color y tipografía, espaciado, componentes,
estados interactivos, navegación y responsive mobile/desktop.
Busco una experiencia de producto refinada, legible y específica:
jerarquía clara, pocas decisiones por pantalla, sin UI genérica
```

```
creada por IA. Primero presenta 2 direcciones visuales y la
justificación. Luego construye la dirección seleccionada.
No inventes branding oficial ni uses datos reales de alumnos.
```

Prompt 3 · Diseñar los flujos y el handoff

```
Conserva estrictamente el design system aprobado.
Crea un prototipo interactivo de estos 3 happy paths:
[PEGAR FLUJOS ELEGIDOS SEGÚN EL RESEARCH].
Para cada pantalla detalla: objetivo del usuario, componentes,
acción, dato mostrado, siguiente paso, estado vacío, loading,
error y confirmación. Haz que las interacciones de demostración
funcionen con datos ficticios, en desktop y móvil.
Al finalizar entrega:
A) inventario de pantallas y componentes;
B) mapa de navegación y diagrama de flujos;
C) especificación de interacciones y criterios de aceptación;
D) entidades de datos sugeridas y relaciones;
E) decisiones visuales/tokens;
F) instrucciones precisas para un agente de código que
   implemente una aplicación Next.js + Supabase + Vercel.
Exporta HTML independiente y, si está disponible, ZIP del proyecto.
No digas que el prototipo es backend funcional.
```

06 · Sesión 1 — Lunes 26 de octubre

Tema: problema → criterio de diseño → sistema visual → happy paths → handoff.

Minuto	Actividad	Cómo facilitarla
00–15	Un alumno presenta su **Grokbot** para automatizar tareas académicas	Mostrar qué automatiza, cuándo se activa y cómo evita errores. Preguntar qué problema resuelve antes de admirar la automatización.
15–20	Recap discovery	Aproximadamente 30 entrevistas; el grupo revisa su documento. Pregunta: «¿qué cambiaremos por lo que aprendimos?».
20–25	Mini-lección: buen gusto	Ver referencias y comparar interfaz genérica vs. diseño con jerarquía e intención.
25–30	Mini-lección: happy paths	Visualizar un flujo de usuario y separar demo de producto real.
30–40	Abrir Claude Design; definir sistema	Screenshots, tokens y componentes.
40–55	Crear tres flujos interactivos	Probar clics y navegación con un compañero.
55–60	Exportar + planificar	Descargar standalone HTML o ZIP; usar el prompt de handoff; **pedir un plan, no implementación aún**.

Cierre antes del miércoles: conservar una carpeta llamada `miu-reimagined/` que contenga `research/`, `design/`, `docs/` y archivos exportados. El objetivo prioritario es tener Claude Design suficientemente terminado, con tres recorridos navegables.

07 · Sesión 2 — Miércoles 28 de octubre

Tema: pasar del HTML/prototipo a un proyecto web real; preparar dependencias y publicar.

Minuto	Actividad	Qué verificar
00–10	Enseñar arquitectura y el flujo GitHub → Vercel → app → Supabase	El grupo sabe qué servicio hace qué.
10–20	Agente revisa research + HTML/ZIP + este PDF y produce plan	Plan con fases, esquema de datos, OAuth, seguridad y pruebas.
20–30	Mostrar cuentas, credenciales y OAuth de Google	Nunca exponer secretos, permisos mínimos, URL callback correcta.
30–45	Laboratorio: crear repo Next.js, Supabase y conectar variables	La app funciona en localhost.
45–60	Implementar primer flujo y publicar URL provisional	Al menos una pantalla funcional; guardar pendientes.

Trabajo autónomo antes del lunes 2: terminar 3 happy paths, asegurar sesión con Google o una alternativa demostrable si hay bloqueo institucional, agregar seed de datos ficticios, probar lectura/escritura, conectar dominio si se compró y publicar URL estable.

08 · Arquitectura recomendada (simple y defendible)

```
Estudiante (navegador)
        |
        v
Vercel: Next.js + UI responsive
  |                  |
  |                  +-- Ruta /auth/callback para iniciar sesión
  |
  +-- Supabase Auth <--- OAuth Google (Google Cloud)
  |
  +-- Supabase Postgres (tablas de prueba + RLS)
        |
        +-- perfiles, cursos, inscripciones,
            tareas, entregas, horarios y anuncios

GitHub → repositorio → despliegues automáticos en Vercel
Namecheap → DNS del dominio propio (opcional)
```

Stack por defecto: Next.js (App Router, TypeScript), Tailwind CSS, componentes React reutilizables, Supabase Postgres/Auth con `@supabase/ssr`, GitHub y Vercel. Si el equipo elige otra tecnología, el plan debe explicar por qué y no agregar complejidad innecesaria.

Cuentas necesarias: Claude (Claude Design; Claude Code si tienen acceso), GitHub, Supabase, Vercel, Google Cloud Console con cuenta Google, y un editor o agente (Claude Code, Codex, Cursor o Antigravity). **Namecheap es opcional y de pago.** Las cuentas de agentes de código pueden requerir plan de pago o tener límites; verificar disponibilidad antes de empezar. La opción gratuita de Vercel genera una URL `*.vercel.app` sin comprar dominio.

Modelo inicial de datos (adaptar tras leer research)

Tabla	Campos mínimos	Relación / regla
`profiles`	`id`, `display_name`, `role`	`id` corresponde al usuario de Supabase Auth.
`courses`	`id`, `name`, `code`, `teacher_name`	Cursos ficticios; datos semilla.
`enrollments`	`id`, `user_id`, `course_id`	Define los cursos del usuario.
`tasks`	`id`, `course_id`, `title`, `description`, `due_at`, `status`	Tareas publicadas ficticias.
`submissions`	`id`, `task_id`, `user_id`, `status`, `submitted_at`	El alumno solo consulta/modifica su entrega.
`class_sessions`	`id`, `course_id`, `starts_at`, `ends_at`, `location`	Horario.
`announcements`	`id`, `course_id`, `title`, `body`, `published_at`	Avisos visibles para inscritos.

Importante: un estudiante no debe ver entregas privadas de otro, ni editar cursos o anuncios administrativos. Las RLS deben implementar estas restricciones y probarse con **dos usuarios distintos**. Evitar construir roles de profesores si no forman parte de los tres flujos priorizados.

09 · Guía paso a paso: de cero a URL pública

Paso 0 — Preparar la computadora y las cuentas

1. Crear cuenta en GitHub, Supabase, Vercel y Claude. Abrir Google Cloud Console.
2. Instalar una versión LTS compatible de Node.js, Git y un editor con terminal (VS Code, Cursor u otro). Confirmar en la terminal: `node -v`, `npm -v` y `git --version`.
3. Crear una carpeta local `miu-reimagined`; dentro guardar el PDF, research y exportación `design/miu-design.html` o su ZIP.
4. Nunca guardar contraseñas, tokens ni Client Secrets en PDF, capturas, ChatGPT o un repositorio público.

Definición de listo: terminal reconoce Node, npm y Git; las cuatro cuentas base están disponibles.

Paso 1 — Terminar y exportar Claude Design

1. Abrir **Claude** → **Artifacts** → **Design** o elegir `Output > Design` dentro de una conversación.
2. Adjuntar referencias + resultados del discovery + Prompt 2. Elegir una dirección y validar el sistema de diseño.
3. Ejecutar Prompt 3; probar cada camino con clics y evaluar móvil.
4. Usar el botón **Export** → **Standalone HTML** y, si conviene, **Download .zip**. Guardar ambos si están disponibles.
5. También existe handoff directo con Claude Code mediante `/design` y sincronización de diseño; usar el export HTML/ZIP como artefacto portable para Codex/Cursor/Antigravity.

Aclaración de versión (octubre de 2026): Claude Design se encuentra integrado en Claude y la versión separada `claude.ai/design` tiene fecha anunciada de cierre el **14 de diciembre de 2026**. Elegir la versión integrada para futuros workshops. Guía oficial.

Paso 2 — Pedir un PLAN antes de generar código

1. Abrir Claude Code, Codex, Cursor o Antigravity sobre `miu-reimagined/`.
2. Adjuntar el research, este PDF/Markdown, el HTML/ZIP, el mapa de flujos y los criterios de aceptación.
3. Pegar el **Prompt 4 (plan maestro)** de la sección 12.
4. Solicitar el plan en `docs/IMPLEMENTATION_PLAN.md` con fases verificables y una lista de preguntas bloqueantes.
5. **Revisar** que el plan: respete el diseño, utilice datos ficticios, configure login, diseñe RLS, haga deploy y contemple pruebas.
6. Aprobar por fases pequeñas. No permitir que el agente ejecute SQL destructivo, publique secretos o contrate servicios sin autorización humana.

Definición de listo: hay un plan aprobado, estructura de carpetas y una tabla de tareas por implementar.

Paso 3 — Crear Supabase

1. Ir a Supabase Dashboard → **New project**. Elegir nombre y región; guardar la contraseña del proyecto de manera segura.
2. Esperar a que se aprovisione. En **Connect** o la configuración de API copiar dos valores: **Project URL** y **Publishable key**. No usar una secret key en el navegador.

3. En el SQL Editor crear las tablas según el plan. No pedir a la IA ejecutar un script incomprensible: revisar cada tabla, clave, relación y privilegio.
4. Activar **RLS** y configurar privilegios/policies para **cada** tabla expuesta. Probar `anon`, estudiante A y estudiante B; una policy permisiva universal es un error de seguridad.
5. Agregar **datos semilla ficticios** para cursos, tareas, clases y anuncios; nunca copiar expedientes académicos reales.

Definición de listo: Supabase creado; esquema y RLS visibles; datos de prueba accesibles con permisos correctos. Documentación: Supabase Next.js y RLS.

Paso 4 — Crear el proyecto Next.js y conectar Supabase

Si **no hay repositorio existente**, el punto de partida más corto que propone Supabase es:

```
npx create-next-app@latest miu-reimagined -e with-supabase
cd miu-reimagined
npm install
npm run dev
```

Si el agente ya creó un proyecto Next.js correcto, **no generes otro encima**. El template `with-supabase` incorpora TypeScript, Tailwind y autenticación con cookies; confirmar sus archivos y adaptar el prototipo visual exportado a componentes React.

Crear/editar `.env.local` en la raíz del proyecto:

```
NEXT_PUBLIC_SUPABASE_URL=https://TU-PROYECTO.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=TU_CLAVE_PUBLICABLE
```

La clave publicable está diseñada para cliente, **pero no concede permiso para omitir RLS**. Nunca subir `.env.local` a GitHub. Si se agrega una clave secreta para funciones administrativas, debe residir únicamente en el servidor, protegida y justificada.

Abrir `http://localhost:3000`. El agente deberá construir componentes desde el HTML, no insertar toda la aplicación como un enorme bloque de HTML sin estructura.

Definición de listo: servidor local responde, estilos correctos, cliente Supabase puede consultar datos de prueba.

Paso 5 — Crear login con Google: OAuth de principio a fin

El vocabulario correcto: para **Google Login** necesitas un **OAuth Client ID + OAuth Client Secret**. **No** necesitas una Google API key genérica. La identidad de Google y la autorización para leer tareas son dos problemas diferentes; Google autentica y RLS autoriza.

A. Configurar Google Cloud

1. Abrir Google Cloud Console y crear/seleccionar un **Project**.
2. Ir a **Google Auth Platform** → configurar **Branding** (nombre reconocible como «MIU Reimagined – Demo»), **Audience** (si corresponde, External) y **Data access/Scopes** mínimos: `openid`, `email`, `profile`.
3. Si la aplicación permanece en estado **Testing**, agregar los correos de prueba autorizados. Algunos correos universitarios pueden estar restringidos por el administrador de Google Workspace.

4. Ir a **Clients** → **Create client** → **Web application**.
5. En **Authorized redirect URIs**, pegar exactamente la callback que presenta Supabase, normalmente:

```
https://TU-PROYECTO.supabase.co/auth/v1/callback
```

1. Guardar **Client ID** y **Client Secret**. **No** poner el Client Secret en el frontend, PDF, repositorio ni en capturas compartidas.

B. Configurar Supabase Auth

1. En Supabase → **Authentication** → **Sign In / Providers** → **Google**, habilitar Google y pegar **Client ID** y **Client Secret en Supabase**.
2. En **Authentication** → **URL Configuration** ajustar:
   - **Site URL**: URL pública principal cuando esté definida (por ahora puede ser tu `*.vercel.app`).
   - **Redirect URLs**: URLs de callback de TU aplicación, por ejemplo `http://localhost:3000/auth/callback`, `https://tu-miulab.vercel.app/auth/callback` y, luego, `https://tudominio.com/auth/callback`.
3. No confundir las dos capas de callback: **Google** → **Supabase** usa `/auth/v1/callback`; **Supabase** → **Next.js** usa `/auth/callback` (si así se programó).

C. Configurar el código Next.js

1. Usar el patrón de **Supabase Auth con SSR + PKCE**, con clientes de navegador y servidor y una ruta `/auth/callback` que canjee el `code` por sesión.
2. Crear botón «Continuar con Google» con `signInWithOAuth({ provider: 'google', options: { redirectTo: ... }})` según la implementación oficial elegida.
3. Proteger rutas privadas y crear un botón de salir; no asumir que ocultar un botón protege la base de datos.
4. Probar login y logout con **dos cuentas Google de prueba**; revisar quién ve los registros de quién.

Definición de listo: login real en `localhost` y URL pública; sesión estable después del refresh; logout; RLS correcta. Guías: Sign in with Google, Redirect URLs, Auth Next.js.

Paso 6 — Llevar el código a GitHub

1. Crear repositorio (p. ej. `miu-reimagined`) en GitHub; por defecto **privado** si todavía hay información que revisar.
2. Desde terminal: `git status`; verificar `.gitignore` con `.env.local`, `.env.*.local`, `node_modules/` y `.next/` excluidos.
3. Subir el proyecto mediante Git o la integración de GitHub del editor. Un flujo habitual:

```
git add .
git commit -m "Initial MIU MVP"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/miu-reimagined.git
git push -u origin main
```

Estos comandos asumen que el repo local y el remoto se configuraron correctamente. Si el agente ya enlazó el repositorio, **no repetir** `git remote add origin`.

Definición de listo: código visible en GitHub, sin secretos, README con cómo ejecutar.

Paso 7 — Publicar en Vercel (Hobby)

1. Abrir Vercel e iniciar sesión con GitHub.
2. **Add New** → **Project** → importar repositorio de GitHub.
3. Confirmar framework Next.js, directorio raíz y comando de build detectados; no modificar sin razón.
4. En **Environment Variables** registrar al menos las dos variables públicas de Supabase. Cualquier secreto de backend va como variable **server-only**, jamás con prefijo `NEXT_PUBLIC_`.
5. **Deploy**. Si la compilación falla, leer logs, corregir localmente con `npm run build` y hacer push de un nuevo commit.
6. Abrir la URL `https://tu-proyecto.vercel.app` en incógnito y desde un teléfono. Verificar login, contenidos y recarga de página.
7. Completar Redirect URLs en Supabase y repetir una prueba de Google Login usando la URL de Vercel.

Definición de listo: app disponible por HTTPS en `vercel.app`. **El dominio propio no es requisito para que exista una URL separada.** Guía Vercel Git.

Paso 8 — Dominio propio en Namecheap (opcional)

1. Elegir un nombre que deje claro que es una **demo independiente**, sin suplantar un dominio institucional. Buscar disponibilidad en Namecheap.
2. Comprar el dominio **solo si el alumno desea asumir el costo**; revisar renovación, privacidad y precio final. No existe un dominio .com gratis incluido con Vercel Hobby.
3. En Vercel → proyecto → **Settings** → **Domains** → **Add Domain** ingresar el dominio.
4. Vercel indicará los registros DNS precisos. En Namecheap → **Domain List** → **Manage** → **Advanced DNS**, crear/editar los registros según la instrucción mostrada:
   - Dominio raíz (`example.com`): típicamente un **registro A**.
   - Subdominio (`www.example.com`): típicamente un **CNAME**.
5. No copiar IPs o valores de tutoriales viejos: **los valores de Vercel para tu proyecto son la fuente de verdad**. Evitar registros en conflicto.
6. Esperar validación de dominio y HTTPS, luego actualizar **Site URL** y **Redirect URLs** en Supabase; no cambiar el callback Google → Supabase si no cambió el proyecto Supabase.
7. Probar login, logout y los tres flujos en la nueva URL.

Definición de listo: dominio personalizado responde por HTTPS; redirecciones correctas. Dominios en Vercel.

Paso 9 — Cerrar el MVP y probarlo antes de salir a la calle

- Crear dataset de prueba sin nombres, correos, calificaciones ni mensajes de alumnos reales.
- Cada uno de los tres flujos debe funcionar **sin explicaciones del constructor**.
- Verificar **persistencia**: refrescar no borra un cambio que debería guardarse.
- Verificar **responsive**: desktop y pantalla móvil.
- Verificar **autorización** con dos usuarios; un alumno no ve datos privados de otro.
- Crear mensajes de loading, vacío, error y éxito en las pantallas críticas.
- Agregar etiqueta visible: **«Prototipo académico no oficial. Sin vinculación operativa con MIU/UFM.»**

10 · Sesión 3 — Lunes 2 de noviembre: salir a la calle

Requisito: traer una URL que ya funcione. Cada equipo debe mostrar la plataforma a **tres estudiantes**. Pueden agendar las pruebas antes del workshop; es preferible porque reduce tiempo perdido buscando participantes.

Guion de la prueba (10–12 minutos por estudiante)

1. **Invitación + permiso (1 min):** «Estamos probando una propuesta estudiantil no oficial. Queremos probar el diseño, no evaluarte a ti. ¿Puedes intentar tres actividades? Si usamos Granola, ¿autorizas que transcribamos esta conversación? Puedes decir que no; tomamos notas manuales».
2. **Contexto (1 min):** «¿Cómo gestionas hoy tareas, horarios y avisos?» No inducir respuestas.
3. **Tarea A (2 min):** «Encuentra qué tarea requiere tu atención primero y dime cuándo vence».
4. **Tarea B (2 min):** «Averigua a qué hora y dónde sería tu próxima clase».
5. **Tarea C (2 min):** «Encuentra el último aviso importante de uno de tus cursos».
6. **Cierre (2–3 min):** «¿Qué fue confuso? ¿Qué esperabas que ocurriera? ¿Qué cambiarías primero? ¿En qué caso usarías esto durante la semana?».

Moderación: no enseñar dónde dar clic; observar y preguntar «¿qué esperabas?». No pedir contraseñas, expediente, número de carné ni uso de cuentas reales de MIU.

Usar Granola sin perder el criterio

Granola tiene app móvil para iOS/Android que permite documentar conversaciones **presenciales**. Antes de empezar: pedir permiso verbal, abrir la app y comenzar la nota conscientemente. Después: revisar el texto generado y corregir cualquier inferencia incorrecta. Si alguien no acepta, **no transcribir** y usar notas manuales. Granola · Privacidad y consentimiento.

Matriz para comparar a los tres participantes

Observación	Estudiante 1	Estudiante 2	Estudiante 3
¿Completó tarea A sin ayuda?	Sí / Parcial / No	Sí / Parcial / No	Sí / Parcial / No
¿Completó tarea B sin ayuda?	Sí / Parcial / No	Sí / Parcial / No	Sí / Parcial / No
¿Completó tarea C sin ayuda?	Sí / Parcial / No	Sí / Parcial / No	Sí / Parcial / No
Mayor fricción observada	Escribir	Escribir	Escribir
Frase textual relevante	Cita autorizada	Cita autorizada	Cita autorizada
Prioridad sugerida	P0 / P1 / P2	P0 / P1 / P2	P0 / P1 / P2

No usar «3 de 3» como validación estadística: es una muestra cualitativa pequeña. Sirve para detectar patrones de fricción y rediseñar con rapidez, no para estimar la satisfacción de toda la universidad.

Cómo priorizar después de las pruebas

- **P0:** impide completar uno de los tres happy paths o deja datos inseguros.
- **P1:** causa confusión importante o afecta a dos o más participantes.

- **P2:** mejora estética o conveniencia con impacto menor.

El equipo implementará **al menos dos mejoras** justificadas por evidencia, verificará que no rompieron otros flujos y hará redeploy antes del miércoles.

Prompt 5 · Síntesis de entrevistas de usabilidad

```
Eres UX Researcher. Te paso las notas autorizadas de 3 pruebas
presenciales de una demo no oficial de MIU.
NO inventes citas, tiempos, porcentajes ni observaciones.
1. Separa observado vs. dicho vs. interpretación del equipo.
2. Construye matriz participante × tarea × resultado.
3. Identifica fricciones repetidas y excepciones.
4. Prioriza P0/P1/P2 por severidad y frecuencia.
5. Propón 3 mejoras pequeñas con criterio de aceptación.
6. Escribe 3 hallazgos para comunicar en clase.
La muestra es cualitativa y pequeña: no generalices.
```

11 · Sesión 4 — Miércoles 4 de noviembre: Demo day

Guion de presentación de 3 minutos

Tiempo	Mostrar	Evidencia
0:00–0:30	«Este es el problema»	Insight extraído de las entrevistas; cita o observación real.
0:30–1:00	«Diseñamos esta experiencia»	Sistema de diseño y decisión clave de UX.
1:00–2:00	**Demo real en URL pública**	Tres flujos; login y lectura/escritura cuando aplique.
2:00–2:40	«Tres alumnos la probaron»	Evidencia de fricciones y mejoras implementadas.
2:40–3:00	«Qué haríamos después»	Una limitación reconocida y próximo paso.

Rúbrica sugerida (100 puntos)

Dimensión	Puntos	Evidencia mínima
Discovery y definición del problema	20	Pain point fundamentado en entrevistas, sin inventar datos.
Experiencia y sistema de diseño	20	UI coherente, jerarquía, responsive y accesibilidad básica.
Funcionalidad de los happy paths	25	Tres flujos reales, persistencia o navegación según corresponda.
Implementación y seguridad básica	15	URL activa, GitHub, Auth, RLS y ausencia de secretos.
Investigación y mejora	20	Tres pruebas, comparación y dos cambios explicados.

Entregables obligatorios: (1) URL pública, (2) repositorio y README, (3) diseño exportado o documentación del design system, (4) investigación de tres usuarios, (5) lista de cambios con evidencia, (6) demo de tres minutos.

12 · PROMPT MAESTRO para Claude Code / Codex / Cursor / Antigravity

Copia el texto completo siguiente al agente **después de adjuntar este PDF, el documento de entrevistas, el HTML/ ZIP de Claude Design y cualquier especificación de flujos**. Primero debe generar un plan, **no ejecutar todo inmediatamente**.

```
ROL
Eres un Senior Full-Stack Engineer con criterio de Product Manager,
UX Designer y especialista en seguridad para prototipos académicos.
Tu alumno es de primer año de universidad: explica cada paso en
español claro, sin omitir decisiones técnicas importantes.

CONTEXTO
Vamos a construir MIU Reimagined, una propuesta académica NO
OFICIAL para mejorar la gestión de tareas, horarios, clases y
comunicaciones de estudiantes de Universidad Francisco Marroquín.
NO tenemos autorización ni acceso a bases de datos internas de UFM.
Nunca intentes conectarte, extraer información ni simular acceso
privilegiado al MIU oficial. Usa exclusivamente datos ficticios.

INPUTS DISPONIBLES
A. PDF/Markdown: Guía Beyond Vibe Coding MIU (esta guía).
B. Documento de discovery: aproximadamente 30 entrevistas reales.
C. Export de Claude Design en HTML independiente o ZIP.
D. Inventario de pantallas, tokens y happy paths aprobados.

OBJETIVO DEL MVP
Aplicación web Next.js + TypeScript + Tailwind + Supabase Auth y
Postgres, hospedada en Vercel Hobby con URL *.vercel.app y dominio
Namecheap solo si se compró, con login Google OAuth.
Tres happy paths verificables de principio a fin, interfaz
responsive y diseño fiel al prototipo, con persistencia cuando
una acción deba modificar datos.

RESTRICCIONES
- Prioriza hallazgos documentados. NO inventes datos ni citas.
- Limita el alcance a tres flujos; registra otros como backlog.
- NO pagues servicios, compres dominios ni hagas despliegues
  públicos o cambios destructivos sin mi autorización.
- NO expongas Client Secret, service-role/secret keys, tokens,
  contraseñas, datos de alumnos ni archivos .env en Git.
- Toda tabla expuesta requiere revisión de grants, RLS y policies.
- Haz login/logout y pruebas de separación entre usuarios.
- No uses Google API key para sustituir OAuth Client ID/Secret.
- Prefiere patrones oficiales y actuales de Supabase SSR/PKCE.
- Si el HTML es un prototipo estático, transpórtalo a componentes
  React mantenibles; NO asumas que ya tiene backend.
- Mantén etiqueta visible «Prototipo académico no oficial».
- No afirmes funcionalidad sin demostrarla con una prueba.

FASE 0 — SOLO PLAN, SIN MODIFICAR EL SISTEMA
1. Lee y sintetiza inputs; marca huecos, decisiones y riesgos.
2. Identifica los 3 flujos y criterios de aceptación concretos.
3. Propón stack, estructura de carpetas y arquitectura simple.
4. Crea esquema de tablas/relaciones con datos ficticios.
5. Diseña roles, grants y RLS con pruebas allow/deny.
6. Diseña Google OAuth con 2 callbacks diferenciadas:
   Google → Supabase /auth/v1/callback
   Supabase → Next.js /auth/callback.
7. Diseña variables de entorno públicas vs. privadas.
8. Descompón trabajo en hitos pequeños con verificación.
9. Incluye despliegue GitHub → Vercel → dominio opcional.
```

```
10. Lista solo preguntas realmente bloqueantes.
11. Escribe docs/IMPLEMENTATION_PLAN.md y una checklist.
12. DETENTE y pide aprobación del plan antes de ejecutar.

FASE 1 — SCAFFOLD Y DISEÑO (tras aprobación)
- Crea o adapta la app Next.js; usa with-supabase cuando convenga.
- Implementa layout, componentes compartidos, tokens y rutas.
- Sigue la UX del HTML y screenshots; valida responsive.
- Muestra claramente qué botones aún no tienen comportamiento.
- Ejecuta npm run build y corrige errores.

FASE 2 — DATOS Y AUTENTICACIÓN (tras aprobación)
- Crea migraciones SQL y datos semilla falsos.
- Añade grants, policies RLS y tests con dos usuarios.
- Integra Auth con Google vía Supabase, usando SSR/PKCE.
- Implementa login, logout, callback, protección de rutas.
- Solicita al humano completar credenciales en los dashboards.
- Implementa queries y cambios persistentes del MVP.

FASE 3 — PUBLICACIÓN Y QA (tras aprobación)
- Actualiza README e instrucciones de GitHub.
- Indica exactamente qué variables añadir en Vercel.
- Ejecuta comprobaciones build, lint si aplica y smoke tests.
- Guía despliegue a *.vercel.app y ajuste redirect URLs.
- Si se compró dominio, explica DNS sin inventar valores.
- Deja lista de errores conocidos y pruebas manuales.

FASE 4 — FEEDBACK Y MEJORAS
- A partir de 3 sesiones de usabilidad, crea tabla de hallazgos.
- Prioriza 2–3 mejoras observables; estima impacto/esfuerzo.
- Implementa solo las mejoras aprobadas.
- Comprueba que los tres happy paths siguen funcionando.
- Genera un changelog y un guion de demo de tres minutos.

FORMATO DE RESPUESTA EN CADA PASO
Objetivo → qué harás → archivos afectos → comandos o clicks
concretos → qué debo ver al terminar → cómo probar → riesgos.
Cuando necesites un valor de cuenta, usa un placeholder y dime
dónde encontrarlo; nunca inventes IDs, claves ni contraseñas.
```

13 · Solución de problemas (consulta rápida)

Síntoma	Diagnóstico probable	Qué hacer
«redirect_uri_mismatch» en Google	Callback Google → Supabase no coincide exactamente.	Copiar callback desde el panel del proveedor Google en Supabase y pegarla en Google Auth Platform → OAuth Client.
«Access blocked» / «app in testing»	Falta test user, consent screen o política Workspace.	Configurar audiencia/test users; probar cuenta Google personal autorizada; no evadir controles institucionales.
Vuelve al login después de entrar	Callback Next.js, cookies SSR o `redirectTo` incompleto.	Verificar `/auth/callback`, URL permitida, intercambio PKCE y ambiente local/público.
Funciona local, no en Vercel	Faltan variables de entorno, config de build o redirects.	Revisar Deployments → Logs y Environment Variables; redeploy tras corregir.
Supabase responde «permission denied»	Grants insuficientes o policy RLS incorrecta.	Revisar privilegios y RLS; **no** desactivar seguridad «para que funcione».
Un usuario ve entregas ajenas	Policy demasiado amplia o consulta sin restricción.	Detener demo pública hasta corregir grants, RLS y pruebas multiusuario.
Dominio aún no funciona	DNS sin propagación, registros duplicados o mala configuración.	Revisar Domains en Vercel, Advanced DNS en Namecheap y esperar verificación.
El diseño funciona pero no guarda	HTML simula interacciones, no está conectado a base de datos.	Implementar operaciones Supabase y comprobar persistencia tras recargar.
IA cambia toda la UI sin pedirlo	Contexto o límites del prompt insuficientes.	Fijar tokens/componentes aprobados; pedir cambios en pasos pequeños y usar Git para revertir.

14 · Costos, límites y responsabilidades

Servicio	Para qué se usa	Condición importante
Vercel Hobby	Hospedaje / URL `vercel.app`	Gratuito para **uso personal no comercial** y sujeto a límites. Adecuado para demo estudiantil personal, no para desplegar un servicio oficial universitario.
Supabase Free	Auth y base de datos	Gratuito con cuotas; en condiciones de baja actividad un proyecto puede pausarse después de ~7 días. Guardar respaldos lógicos del trabajo (migraciones/seed).
Claude Design	Diseños y prototipos	Disponible en planes Free y otros, con posibles límites de uso; export HTML y ZIP según las opciones de la cuenta.
Claude Code / Codex / Cursor / Antigravity	Asistencia al desarrollo	Requisitos de acceso, créditos y precios cambian; confirmar en la cuenta. No asumir disponibilidad ilimitada gratuita.
GitHub	Versionar código	Repositorio gratuito y flujo de cambios, sujeto a su plan.
Google Cloud OAuth	Login Google	Configurar cliente OAuth y consentimiento; no equivale a acceso administrativo a una cuenta institucional.
Namecheap	Comprar dominio	**Pago opcional**, precio variable y renovación futura.
Granola	Entrevistas y notas	Requiere transparencia/consentimiento; verificar límites de plan y privacidad.

Uso de datos: ninguna tarea necesita expedientes reales, contraseñas de MIU, mensajes privados ni información personal de compañeros. Informar claramente que es una demostración sin autorización institucional. Si el equipo usará logotipos o marca UFM, confirmar permisos y optar por identidad diferenciada en caso de duda.

15 · Biblioteca de recursos y videos

Los videos son **apoyo**, no sustituto de las guías oficiales, que tienen prioridad ante cambios de interfaz. En algunos casos el tutorial es de una versión anterior de Next.js; seguir los conceptos pero usar la documentación actual para configurar la cuenta.

Videos de YouTube con enlaces directos

Paso	Recurso	Enlace
Diseño (ES, 2026)	**Claude Design 2.0** — Claridad Artificial: sistema de diseño y sincronización	Ver video
Diseño (ES, 2026)	**Cómo usar Claude Design desde cero** — Margot Duek: dirección visual y demo de app	Ver video
Diseño/handoff (EN)	**Claude Design: The Complete Guide** — UI Collective: sistema + Claude Code	Ver video
GitHub (ES)	**Git y GitHub desde cero** — MoureDev (usar los capítulos de intro y push)	Ver video
Supabase + Google (EN, 2025)	**Authentication with Supabase + Next.js 16** — guía de OAuth, sesiones y SSR	Ver video
Supabase CRUD (EN, 2025)	**Next.js CRUD App with Supabase** — auth, tablas y operaciones	Ver video
OAuth (ES, 2024)	**Google/Facebook OAuth con Supabase** — Leonardo Jose; centrarse en Google	Ver video
Deploy (EN, 2025)	**Deploy Next.js to Vercel** — AyyazTech: GitHub, env y dominio	Ver video
Deploy breve (EN, 2025)	**Next.js to Vercel** — Codevolution	Ver video
Dominio (EN, 2024)	**Namecheap Domain to Vercel** — Evan Manza	Ver video

Documentación oficial de consulta prioritaria

- Claude Design: ayuda oficial en español
- Supabase: quickstart Next.js
- Supabase: autenticación con Next.js
- Supabase: Google OAuth
- Supabase: URLs de redirección
- Supabase: Row Level Security
- Vercel: publicar desde Git
- Vercel: configurar dominio
- Vercel: condiciones Hobby
- Supabase: pausa del plan Free
- Google: OAuth 2.0 para aplicaciones web
- Granola: consentimiento en conversaciones

16 · Checklist final para copiar al README

```
[ ] Leímos el research original de ~30 estudiantes.
[ ] Definimos 1 problema principal respaldado por evidencia.
[ ] Tenemos 3 happy paths con criterios de aceptación.
[ ] Construimos y documentamos un design system coherente.
[ ] Exportamos el HTML/ZIP de Claude Design.
[ ] El agente preparó un plan y lo aprobamos antes de ejecutar.
[ ] Creamos repositorio GitHub (sin secretos).
[ ] Next.js corre localmente y npm run build no falla.
[ ] Supabase tiene tablas, datos ficticios, grants y RLS.
[ ] Google OAuth funciona en la URL pública (login/logout).
[ ] Dos usuarios no pueden ver datos privados del otro.
[ ] Los 3 recorridos funcionan en computadora y móvil.
[ ] La app está disponible por HTTPS en *.vercel.app.
[ ] Si hay dominio propio: DNS y callbacks funcionan.
[ ] Probamos con 3 estudiantes y documentamos observaciones.
[ ] Implementamos 2 mejoras basadas en lo observado.
[ ] Publicamos la última versión y verificamos la URL.
[ ] El producto informa que NO es el MIU oficial.
[ ] README, URL, investigación y demo final están listos.
```

> **Principio final:** no ganamos por escribir más prompts ni por producir más pantallas. Ganamos cuando un estudiante puede solucionar mejor un problema, con una experiencia que funciona y una decisión de producto respaldada por evidencia.

Material preparado para Beyond Vibe Coding — Workshop MIU · Revisión de enlaces y flujo técnico: 25/oct/2026.